

AlchemyX
AlchemAI

Agentic Research Assistant for intelligent Document Search

SLIIT Codefest 2026

AI Competition

Sub-track 1C: Searching the way a human does

Submission Report

YouTube Demo Video

<https://youtu.be/OWNbHqt6VbA>

Team Members

Shihaz Shaheem

Sisindu Hemasinghe

Sakitha Palliyaguru

Lowaga Rajapaksha

1. Problem Statement & Chosen Sub-track

Problem Context

Enterprise documentation is often fragmented across mixed formats, with relevant facts spread across multiple sources that may be incomplete or conflicting. The challenge is therefore to retrieve evidence and determine whether it is sufficient and reliable enough to support an answer.

The Ashen Era Archive contains 415 fictional documents across about 1,277 pages. Because external model knowledge is irrelevant, answers must be derived entirely from the corpus, whose interconnected sources can also disagree in reliability.

Chosen Sub-track 1C: Searching the Way a Human Does

We selected **Sub-track 1C: Searching the Way a Human Does**, which requires iterative retrieval rather than a single lookup. AlchemAI evaluates retrieved evidence, identifies what is missing, and performs targeted follow-up searches until enough information is available.

This is necessary because one-shot RAG can fail when evidence is incomplete, distributed, conflicting, or drawn from sources with different reliability.

Our Response -AlchemAI

AlchemAI is an agentic research assistant that performs iterative retrieval rather than a single search. Search → Retrieve → Assess → Identify Missing Evidence → Re-query → Resolve → Answer

Its core capabilities include:

- Hybrid semantic + lexical retrieval
- Reranking and evidence accumulation
- Sufficiency checking
- Entity-aware conflict resolution
- Targeted follow-up queries
- Evidence-grounded answers with traceable sources

Its goal is to refine each search using previously retrieved evidence and stop only when the available evidence supports a reliable answer.

2. Solution Overview & Architecture

AlchemAI uses an agentic RAG architecture that separates ingestion, retrieval, reasoning, and answer generation. The corpus is indexed in advance, while each query goes through hybrid retrieval, reranking, sufficiency checking, conflict resolution, and follow-up search before producing a grounded answer.

then prioritize the most relevant evidence before reasoning.

3. Persistent Indexing

Decision: Build document embeddings and BM25 indexes once, then reuse them at runtime.

Why: Re-embedding the full corpus for every query would be wasteful and unnecessary.

4. Iterative Search instead of One-shot RAG

Decision: Use an agent loop with sufficiency checks and targeted follow-up queries.

Why: Track 1C specifically requires the system to learn from retrieved evidence and search again when something is still missing.

5. Evidence Accumulation

Decision: Preserve useful evidence across iterations rather than replacing it every search.

Why: Some answers only become clear after several searches, especially when one source points toward another.

Why: The challenge archive intentionally contains sources with different reliability levels.

8. Structured LLM Outputs

Decision: Use structured JSON for sufficiency, conflict analysis, and follow-up decisions.

Why: Structured state makes the agent loop deterministic enough to

control, validate, test, and expose cleanly in the UI.

9. Graceful Failure Handling

Decision: Add retries, malformed-JSON handling, and API fallback behavior.

Why: External APIs can fail, overload, or return malformed outputs, and the demo must not crash.

10. Entity-aware Conflict Resolution

Decision: Validate that evidence actually refers to the requested entity before using it in adjudication.

Why: Similar names such as different locations or artifacts can otherwise create false conflicts.

4. What Works, What Changed & Current Limitations

What Works	What We Changed During Development	Current Limitations
Hybrid retrieval: combines Voyage/Chroma semantic search with BM25 and RRF fusion for broader recall.	Premature stopping: early versions treated contested evidence as sufficient; adjudication-aware sufficiency and follow-up searching were added.	Synchronous execution: the current pipeline runs research stages sequentially in one request rather than using parallel/asynchronous execution.
Iterative search: accumulates evidence across iterations and generates targeted follow-up searches when evidence is incomplete.	Entity drift: similar entities created false conflicts; chunk-level validation and exact-match boosting were introduced.	External service dependency: AlchemAI relies on Voyage AI and OpenRouter, so provider availability, quotas, and upstream errors can affect execution.
Entity-aware reasoning: validates exact/alias/related/mismatched evidence to reduce false conflicts from similar names.	Duplicate corroboration: repeated PDF/DOCX copies were initially counted independently; source-family normalization fixed this.	Authority inference is heuristic: source reliability is inferred from text and metadata rather than a fixed verified hierarchy.

Conflict resolution: compares competing claims and source authority instead of immediately treating disagreement as unresolved.	Strict JSON parsing: malformed structured LLM output could crash the agent; parsing, normalization, repair attempts, and safe fallbacks were added.	Retrieval noise: large, mixed corpora can still surface unrelated or similarly named documents.
Source-family deduplication: PDF and DOCX versions of the same work are treated as one underlying source.	Conflict-state and source-role inconsistencies: backend/UI disagreement was corrected using canonical conflict states and clearer evidence roles.	Finite search depth: the agent currently has a maximum of four iterations.
Grounded answers and traceability: final answers are generated from retrieved evidence while readable source labels are shown in the UI.	Follow-up query drift: generated queries sometimes omitted the original entity; follow-up generation now preserves it.	Deployment dependency: scanned-PDF ingestion requires external OCR tools such as Tesseract and Poppler to be installed and configured

5. AI Usage Disclosure

AI tools (Claude, ChatGPT, Codex, Gemini) were used throughout development for code assistance, debugging, architecture review, and edge-case testing, but every core design decision was made and tested by the team, not accepted from AI output. For example, sufficiency logic was designed by first using Gemini to surface failure modes ("premature sufficiency," infinite re-query loops) before any code was written, and a retrieval bug — a single embedding function being reused for both document storage and query search, silently degrading relevance — was independently diagnosed and fixed by the team. The frontend was initially built in React, then deliberately converted to Streamlit after the team judged that building a separate API bridge was too high-risk given the timeline. Full, unmodified AI interaction logs for all four members are included in `ai_usage/` as required.

Team Member	Main Contribution
Shihaz Shaheem	Designed and implemented the retrieval pipeline, including semantic search, BM25, RRF fusion, Voyage reranking, and iterative re-query behavior. Contributed to retrieval evaluation and integration with the agent loop.
Sisindu Hemasinghe	Developed the reasoning layer, including sufficiency assessment, stopping decisions, conflict detection/adjudication, and LLM-based control flow.
Sakitha Palliyaguru	Developed the Streamlit interface, research timeline and evidence views, demo workflow, documentation, and repository/presentation support.
Lowaga Rajapaksha	Handled corpus ingestion and preprocessing, including document parsing, chunking, metadata handling, OCR support, and index preparation.

Shared Contribution: All team members contributed to integration, testing, debugging, evaluation, and validation of the final system.